

Main contributions

- Set up and tuned AR tracking with ARFoundation and reference image targets so the book pages could be detected reliably.
- Helped build and debug the runtime AI 3D pipeline (CameraCapture → Meshy → glTF loading) and understood how the whole chain fit together.
- Implemented and adjusted sound design in both AR and VR to make the experiences feel more alive and immersive.

Reflections

When I look back on this course, I can honestly say the projects taught me XR in a way lectures never could. Before, I could define AR and VR in a neat way, but now I understand them through all the small wins and annoying bugs we had. AR became “make digital stuff behave like it belongs in the real world,” and VR became “make the user feel like their body is part of the game.” XR is just the big umbrella over that, and I get now why the differences matter so much when you’re actually building things.

In the AR project I mainly owned tracking and the experience side (especially sound). Our idea was to take a physical fantasy book and make it feel alive. That’s the kind of use case where AR makes real sense: you’re adding something to an object people already care about, not just showing a random model on a screen. The moment a page is recognized and a creature pops up in the right spot is exactly what gives value to the end user. But I also learned fast that AR only feels magical if tracking is solid. If the model drifts or jumps, users stop trusting it instantly. ARFoundation makes marker-based tracking approachable, but it still depends heavily on real-world conditions like lighting, viewing angles, and whether your reference images are actually good targets. Working with that made the tracking theory feel very real: it’s not a background feature, it’s the foundation of the whole app.

One small AR problem that taught me a lot was our orientation issue. Tracking worked, but the animal kept spawning with its back turned. Fixing it using a parent GameObject was a simple change, but it forced me to understand transforms properly. In AR, tiny rotation mistakes look huge because everything is anchored to the real world. After sorting that out, I felt more confident about owning the scene logic instead of treating Unity like a black box.

Sound in AR was another place where I changed my mindset. I used to think audio was just “nice to have,” but AR immersion really benefits from quick, clear feedback. When kids tap the creature and it reacts with sound, the experience clicks. We didn’t need fancy tech for that, just the right kind of feedback tied to the physical context. That’s something I’ll carry forward: XR presence often comes from simple things done well, not from piling on features.

The big turning point in AR was pivoting to Meshy and runtime image-to-3D. My first reaction was basically “this might be too ambitious.” We already had a baseline working, and switching direction late is risky. But I’ve come to respect the decision a lot. The course talks about XR moving toward dynamic, context-aware content, and this pivot forced us to try that in practice. Even though it raised the risk, it made the project much more aligned with what modern AR is aiming toward.

Building the runtime pipeline was where I felt the most ownership. It wasn’t drag-and-drop anymore. We had to think in a strict flow: capture a clean image, send it to an external service, wait for generation, download a GLB, then load it into Unity at runtime. If one part failed, everything failed. That gave me a stronger sense of XR architecture and why runtime content is still tricky. It was cool to work on, but also fragile in a way that normal Unity projects usually aren’t.

The hardest AR issue we didn’t fully solve was textures on the generated models. We got the models to load after updating glTFast, but they came out purple and untextured. Trying to fix that showed me how deep AR rendering goes: shaders, materials in glTF, Android build quirks, and runtime limitations all interact. It also connected straight to what we discussed in class about AR challenges today: consistency across devices and pipelines is still a real struggle. Even though we didn’t get it polished in time, I learned a lot from chasing the problem and seeing how many layers are involved.

Moving into VR was a shift in mindset. AR is about alignment with reality, VR is about believable presence inside a made-up world. Our VR game relied on 6DOF interaction more than anything else. Grabbing a gun, inserting a mag, aiming and shooting with your hands is the core VR advantage. It’s not just a cool feature; it’s what makes the experience different from a flat-screen game. Watching people naturally understand those actions made the 6DOF theory feel obvious instead of abstract.

My main focus in VR was sound and combat feedback. Since we were doing zombies and horror vibes, audio wasn’t optional. Without sound cues, the enemies don’t feel scary, and the player doesn’t feel tension. I worked on shooting sounds, enemy cues, and background atmosphere to push immersion. I also helped with hit feedback, especially making blood only appear on actual hits. For me, that was about timing and cause-effect. In VR your body notices instantly if feedback comes too early or too late, and it breaks the feeling of control.

Quest development also made me respect performance theory more. On desktop you can hide lag behind settings, but on Quest frame drops kill immersion and can make people feel sick. That’s why we kept prioritizing a stable core loop and cut anything that didn’t support the horror feel, like the time-warp idea. It worked technically, but it didn’t fit the experience we wanted, so removing it was the right call. That decision taught me that good XR design is protecting a clear user experience, not cramming in every possible mechanic.

On the teamwork side, I think I got better at jumping into tasks and keeping momentum. I often started things fast, then we solved conflicts together when code or ideas clashed. Over time we got quicker at prototyping and less scared of rough first versions. That “build it, test it, fix it, repeat” approach is something XR almost forces you into, because you can’t predict everything on paper.

If I'm honest about the final result, both apps are solid prototypes. The AR app has a strong concept and clear value. With more time, caching generated Meshy models so they can be reused would be a big next step for both performance and user flow. The VR app has a complete loop with physical interaction, enemies, and feedback, and it shows that we understood what makes VR special.

Overall, this course changed how I see XR. I don't think of tracking, rendering, interaction, and immersion as separate topics anymore. They're one connected system, and the user feels the weakest link immediately. I also feel way more confident owning pipelines and making design choices based on XR strengths, not just on what looks cool. The biggest takeaway for me is technical confidence in building XR systems that run on real hardware, and personal confidence in taking on ambitious ideas even when they feel risky.